Putting v2 on the ZedBoard: the complete guide

This guide takes you from the files on your PC to a running board and the slide-24 measurements. It collects everything in one place; the shorter step list is GETTING_STARTED.md.

There are two ways to run v2:

	Way 1: FPGA only	**Way 2: ARM + FPGA**
what runs	the controller on the programmable logic; a small boot sequencer does the set-up	the controller + the Zynq ARM running the C console program
software	none	Vitis app (`sw/`)
records go to	a 4 KB block RAM	DDR memory, through the HP0 port
use it for	first bring-up, the flash-ID and temperature demos, crossbar switching, timing and area	everything, including the CPU-load measurement and the programs (`se ...`)
time	about 45 minutes	1-2 hours

Do Way 1 first. If Way 1 works, the hardware, the Pmods and your cables are right, and Way 2 only adds the ARM side.

---

1. What you need

Hardware

item	**needed for**	**notes**
ZedBoard (Rev C or D) with its 12 V power supply	both	
2 micro-USB cables	both	**J17 (PROG)** = JTAG for programming; **J14 (UART)** = the ARM console (Way 2)
Digilent **PmodUSBUART** + 1 micro-USB cable	Way 1 PC link (and `flowa` in Way 2)	on **JA, top row**
Digilent **PmodSF3** (serial flash)	both	on **JB**, the whole 12-pin connector
Digilent **PmodTMP2** (temperature, I2C)	both	on **JC**, see 3.3
logic analyser, 8 channels, >= 24 MS/s (an FX2-based one with PulseView is enough) + jumper wires	measurements	on **JD**
two 4.7 kOhm resistors	only if I2C reads fail	pull-ups from JC3 / JC4 to 3.3 V

Software (once)

tool	why
Vivado + Vitis, 2020.2 or newer (free licence covers the XC7Z020), with **cable drivers**	build and program
ZedBoard board files (Vivado: Tools -> Vivado Store -> Boards -> Avnet ZedBoard)	Way 2 block design
Python 3 + `pip install pyserial`	`tools/pc_frame.py` (the PC side of the flash-ID demo)
Tera Term or PuTTY	the Way 2 console
PulseView (sigrok)	the logic analyser
optional: WSL/Linux with `iverilog`, `yosys`	the fast simulation scripts

Unzip the project to a short path without spaces, e.g. `C:\ssc\smart_serial_controller_v2` (GETTING_STARTED step 1).

---

2. Before the board: simulate and build

1. **Simulate** (proves the Verilog works, no board needed): `windows\1_simulate.bat` (Vivado, 15-40 min) or `sh sim/run_unit.sh` and `sh sim/run_system.sh` (WSL/Linux). Every test must end with `ALL n CHECKS PASSED`.
2. **Build the Way 1 bitstream**, either
   - batch: double-click `windows\2_build_way1_fpga_only.bat` (10-20 min), or
   - GUI: double-click `windows\0_create_project_gui.bat`, then in the Flow Navigator **Run Synthesis -> Run Implementation -> Generate Bitstream**.
3. **Check the build before you go to the board:**

check	where	must be
timing	batch: last lines of the log; GUI: Open Implemented Design -> Report Timing Summary	WNS >= 0, TNS = 0, WHS >= 0, "All user specified timing constraints are met"
I/O	GUI: Report I/O, or `build\standalone\utilization.rpt`	about 50 bonded I/O, all with a pin (no "unconstrained" warning)
DRC	Messages tab	only warnings; **ZPS7-1** "PS7 block required" is expected in Way 1
bitstream	`build\standalone\zed2_top_standalone.bit` (batch) or `build\gui\ssc2_gui.runs\impl_1\zed2_top_standalone.bit` (GUI)	exists

> If WNS < 0, see TIMING.md before programming: a design that fails timing may still seem to work, but it is not a valid result.

---

3. Prepare the board (power OFF)

3.1 Jumpers

jumper	setting	why
JP7-JP11 (boot mode, near the SD card slot)	all on the **GND** side	boot from JTAG: the board waits for Vivado / Vitis
J18 (VADJ)	**2.5 V** (the default)	the switches and buttons are constrained as LVCMOS25
PmodUSBUART **JP1**	**LCL**	the Pmod is powered by the ZedBoard

3.2 Switches and buttons

All eight switches **down** (SW0-SW7 = 0) to start. BTNC is the reset.

3.3 Pmods (pin 1 to pin 1: pin 1 is marked on the board and on the Pmod)

connector	Pmod	how
JA	PmodUSBUART	top row (JA1-JA4)
JB	PmodSF3	the whole 2 x 6 connector (it uses JB1-4 for SPI and JB7-10 for RST#, WP#, HOLD#)
JC	PmodTMP2	its 2 x 4 header in the four columns nearest the GND/VCC end: pins **3-6 and 9-12**, so SCL = JC3 and SDA = JC4; both address jumpers open (address 0x4B)
JD	logic analyser (optional)	JD1-JD4 = whatever SW2..0 selects; JD7 = UART TXD, JD8 = UART RXD, JD9 = I2C SCL, JD10 = I2C SDA (copies, always on); analyser GND to a JD GND pin

Plug and unplug Pmods only with the power off.

3.4 Cables

J17 (PROG) to the PC; the PmodUSBUART's micro-USB to the PC; 12 V power; for Way 2 also J14 (UART). Switch the board on with SW8 (POWER). The green POWER LED lights.

On the PC, note the COM ports (Windows Device Manager -> Ports): the PmodUSBUART appears as a "USB Serial Port" (FTDI); J14 appears as another COM port (the Way 2 console).

---

4. Way 1 on the board

4.1 Program the FPGA

1. Vivado -> **Open Hardware Manager** -> **Open Target** -> **Auto Connect**. The Hardware window shows `xc7z020_1` (and `arm_dap_0`).

2. Right-click `xc7z020_1` -> **Program Device**, choose the `.bit` file (no debug-probes file is needed) -> **Program**.
3. The blue **DONE** LED lights.
4. Press **BTNC** once (reset). The boot sequencer sets up the controller in a few microseconds, then reads the switches every 10 ms.

4.2 Check that it runs (LEDs)

LED	correct behaviour	if not
LD4	blinks once per second (heartbeat)	not programmed, or held in reset
LD0-LD3	count in binary; LD0 changes every 0.4 s (records / 4; 10 records per second)	the hub's temperature reads fail: see LD6
LD5	toggles every 3.2 s (one batch = 32 records)	
LD6	stays **off**	on = a hub request failed: PmodTMP2 position (JC3/JC4), address jumpers, pull-ups
LD7	interrupt line (short pulses)	

4.3 Flash ID through the network (slide 17)

All switches down (SW7 = 0: FRAME mode, frames go to the SPI flash). On the PC, with the PmodUSBUART's COM port:

```
python tools\pc_frame.py COM5 01 9F 03
sent : 01 9F 03
reply: 20 BA 19              <- JEDEC ID of the Micron flash on the PmodSF3
```

`01 9F 03` = "write 1 byte (9F = READ ID), then read 3". The bytes go PC -> UART engine -> UART network interface -> 2 router hops -> SPI network interface -> SPI engine -> flash, and the answer comes back the same way. No CPU is involved. Other frames:

```
python tools\pc_frame.py COM5 04 03 00 00 00 10     # read 16 bytes at address 0 (FF if erased)
```

4.4 Temperature through the network

Set **SW7 up** (ADDRESSED mode: the first byte of a frame says which node it is for), then

```
python tools\pc_frame.py COM5 05 4B 01 00 02 --temp
reply: 0C 80
temperature: 25.0000 C        <- your room temperature
```

Touch the TMP2 and run it again: the value rises.

4.5 Pin crossbar (slide 14)

SW2..0 choose what drives JD1-JD4 (watch them on the logic analyser):

SW2..0	JD1-4 show
000	nothing (off)
001	a copy of the UART engine's pins
010	a copy of the SPI engine's pins (the flash bus): run `pc_frame.py ... 01 9F 03` and decode SPI
011	a copy of the I2C engine's pins
100	SM0 = the UART **program** of slide 13: JD2 carries the temperature bytes every 100 ms, decode as UART 115200 8N1
101	SM1
110	GPIO

Changing the switches while a frame is running never cuts the frame: the crossbar waits for the end of the frame, then hands the pins over.

4.6 What to write down (Way 1)

- WNS, LUTs, FFs, BRAM, power from the Vivado reports (TIMING.md section 6, MEASUREMENTS.md section 5);
- the flash ID and temperature replies;
- from the logic analyser: UART bit time on JD7 (8.68 us at 115 200 baud), SCL period on JD9 (10 us at 100 kHz), the SPI clock on JD (SW = 010).

---

5. Way 2 on the board (ARM + FPGA)

5.1 Build the hardware

Double-click `windows\3_build_way2_arm_fpga.bat` (15-30 min). It creates the block design (Zynq PS + the controller + HP0 for the DMA writer), the bitstream and `build\ps_system\ssc2_system.xsa`. Check that the log ends with WNS >= 0 and `SUCCESS`.

5.2 Build the software (Vitis)

Copy all eight files of `sw\` into the application's `src` folder (the exact clicks for the new and the classic Vitis are in GETTING_STARTED step 5b):

1. platform from `ssc2_system.xsa`, standalone, `ps7_cortexa9_0`;
2. empty C application on that platform;
3. copy `main.c ssc2_driver.c ssc2_driver.h ssc2_regs.h se_demos.c se_demos.h se_programs.h oled_font.h` into `src`, build.

5.3 Run

1. Terminal (Tera Term / PuTTY) on the **J14** COM port, 115200 8N1.

2. Vitis -> Run (with "Program FPGA" ticked, or program the Way 2 bitstream first).

3. The console prints

```
=== Smart Serial Controller v2 - Way 2 (ARM + FPGA) ===
controller v2 found at 0x43c00000
ssc2>
```

4. Type the commands. Correct answers:

command	correct result	slide
`info`	ID 0x53534302, VERSION 0x00020000, JA=UART JB=SPI JC=I2C	
`id`	`JEDEC ID: 20 ba 19 (Micron - PmodSF3)`	v1 path
`temp`	`+25.0000 C` (room temperature), CPU load about 13 accesses + 2 interrupts	20, v1 cost
`hub`	every 3.2 s a batch of 32 records, t about 100 000 us apart; CPU load for 32 readings: 2 accesses, 1 interrupt	18-20
`flowa` + `pc_frame.py COMx 01 9F 03` on the PC	`20 BA 19` on the PC	17
`switch`	waits for the SPI frame (SW_CYCLES about 30 000), "HELLO FROM SM0 ON JD" on JD2; idle switch SW_CYCLES = 2	14
`se uart`, `se spi`, `se i2c`, `se man`	PASS / `20 ba 19` / temperature / 32 bytes, CRC equal, PASS	12-13, measurement 1
`se oled HELLO`	the OLED shows the text and the temperature	

Every command in detail: VERIFICATION.md section 6.

---

6. The slide-24 measurements

#	measurement	how (details in **MEASUREMENTS.md**)
1	flexibility	Way 2: `se uart`, `se spi`, `se i2c`, `se man` one after another, same bitstream
2	speed	logic analyser on JD: UART bit time, SCL period (100 / 400 kHz), top SPI clock of the fixed engine and of the program
3	switch time	`switch`: SW_CYCLES / SW_EDGE counters; JD on the analyser
4	CPU load	`temp` (v1 way) against `hub` (v2 way); the controller counts its own register accesses and interrupts
5	area	Vivado `utilization_hier.rpt`: LUT / FF / BRAM per block
6	NoC scalability	`windows\5_noc_study_vivado.bat` (real LUTs and Fmax) + `noc_study/results.md`

Optional, power: a power meter in the 12 V line; compare the board power with the ARM busy (v1 way, `temp` in a loop) and idle (v2 way, `hub`).

---

7. Optional: start without the PC (SD card)

JTAG loading is enough for the project. To start Way 2 from an SD card: in Vitis create a boot image (`BOOT.BIN` = FSBL + bitstream + `ssc2_app.elf`; Vitis: *Create Boot Image*), copy it to a

FAT32 SD card, and set the boot jumpers to SD mode (JP9 and JP10 to 3V3, JP7, JP8, JP11 to GND; check the boot-mode table printed on the board). Way 1 has no ARM software, so on a Zynq it also needs an FSBL to load its bitstream at power-up; for Way 1 simply use JTAG.

---

8. Troubleshooting on the board

symptom	**cause / fix**
Hardware Manager finds no target	J17 cable (not J14); cable drivers installed?; board powered?; boot jumpers on GND
DONE LED does not light	wrong bitstream (for another device), or programming failed: look at the Tcl console
LD4 does not blink	press BTNC; check the clock constraint (`zed2_standalone.xdc`) is in the project
`pc_frame.py` gets no reply	use the PmodUSBUART's COM port (not J14); Pmod on JA **top** row; JP1 on LCL; SW7 down for `01 9F 03`; press BTNC
flash ID `FF FF FF` or `00 00 00`	PmodSF3 turned round or not on JB
LD6 on, LD0-3 do not count	TMP2 not at JC3/JC4, address jumpers not open, or no pull-ups: add 4.7 kOhm from JC3 and JC4 to 3.3 V
temperature reply but a strange value	the reply is the raw 16-bit register (`0C 80` = 25.0 C); `--temp` converts it
Way 2 stops at "reading the controller ID"	FPGA not programmed: tick "Program FPGA" in Vitis
`ERROR: ID ...` in Way 2	wrong bitstream, or the controller is not at 0x43C0_0000 in the Address Editor
timing failed in the build	TIMING.md

Other problems: GETTING_STARTED.md "Troubleshooting" and VIVADO_GUI.md.